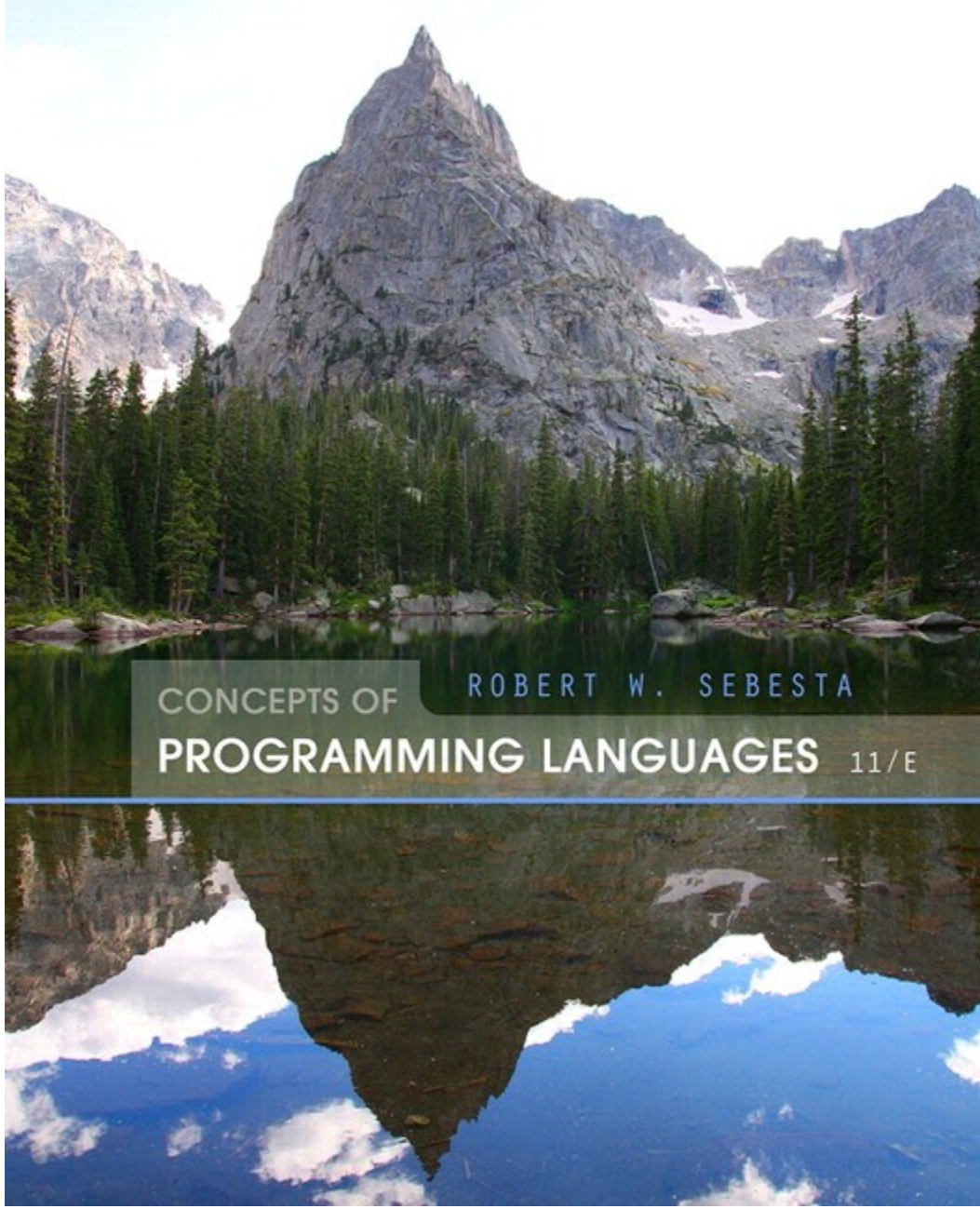
Bölüm 3

Söz Dizimi (Syntax) ve Anlam Bilgisini (Semantics) Tanımlama



ISBN 0-321-49362-1

Bölüm 3 Konular

- 3.1 Giriş
- 3.2 Söz Dizimini (Syntax) Tanımlamanın Genel Sorunu
- 3.3 Söz Dizimini (Syntax) Tanımlamanın Biçimsel Yöntemleri
- 3.4 Özellik Dil Bilgisi (Attribute Grammars)
- 3.5 Programların Anlamlarını Açıklama: Dinamik Anlam Bilimi (Dynamic Semantics)

Copyright © 2015 Pearson. All rights reserved.

3.1 Giriş

- **Söz Dizimi (Syntax):** ifadelerin, deyimlerin ve program birimlerinin biçimi veya yapısı
- **Anlam Bilimi (Semantics):** ifadelerin, deyimlerin ve program birimlerinin anlamı
- Syntax ve semantics bir dilin tanımını sağlar
  - Bir dil tanımının kullanıcıları
    - Diğer dil tasarımcıları
    - Uygulayıcılar
    - Programcılar (dilin kullanıcıları)

Copyright © 2015 Pearson. All rights reserved.

3.2 Syntax Tanımlamanın Genel Sorunu: Terminoloji

- *Cümle (sentence)*, bir alfabe üzerindeki karakterlerin dizisidir.
- *Dil (language)*, bir dizi cümleden oluşur.
- Bir *sözcük birimi (lexeme)*, bir dilin en alt düzeydeki sözdizimsel birimidir. (örneğin: `*`, `sum`, `begin`)
- *Token*, bir lexeme kategorisidir (örneğin, tanımlayıcı, identifier).

Copyright © 2015 Pearson. All rights reserved.

3.3 Dillerin Biçimsel Tanımı

- **Tanıma Aracı (Recognizers)**
  - Bir tanıma aracı, dilin alfabesi üzerinden girdi dizilerini okur ve girdi dizilerinin dile ait olup olmadığına karar verir.
  - Örnek: derleyicinin söz dizimi analizi bölümü

    - Söz dizimi analizinin ayrıntılı tartışması şu kaynakta yer almaktadır:

      Bölüm 4

- **Üreteçler (Generators)**
  - Bir dilin cümlelerini üreten cihaz
  - Belirli bir cümlenin söz dizimi açısından doğru olup olmadığını, onu üretecinin yapısı ile karşılaştırarak belirleyebiliriz.

Copyright © 2015 Pearson. All rights reserved.

3.3.1 BNF ve Context-Free Grammars

- Context-Free Grammars
  - 1950'lerin ortalarında Noam Chomsky tarafından geliştirilmiştir.
  - Doğal dillerin sözdizimini tanımlamak için kullanılan dil üreticileridir.
  - Bağlamdan bağımsız diller (context-free languages) olarak adlandırılan bir dil sınıfını tanımlayabilirsiniz.

- Backus-Naur Form (1959)
  - Algol 58'in sözdizimini tanımlamak için John Backus tarafından icat edildi.
  - BNF, bağlamdan bağımsız dilbilgisine (context-free languages) eşdeğerdir.

Copyright © 2015 Pearson. All rights reserved.

3.3.1.1 BNF'nin Temelleri

- BNF'de, soyutlamalar söz dizimsel yapıların sınıflarını temsil etmek için kullanılırlar – söz dizimsel değişkenler (*nonterminal semboller* veya sadece *terminaller* olarak da adlandırılır) gibi davranırlar.

- *Terminaller*, lexeme'ler veya token'lardır.

- Left-hand side (LHS)'nin kuralı, bir nonterminal'dir, ve a right-hand side (RHS)'nin kuralı, bir dizi terminal ve/veya nonterminal'dir. (A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals)

Copyright © 2015 Pearson. All rights reserved.

3.3.1.1 BNF'nin Temelleri

- Nonterminals genellikle köşeli parantezlerle çevrelenir.
  - BNF kurallarına örnekler:

```
<ident_list> → identifier | identifier, <ident_list>
<if_stmt> → if <logic_expr> then <stmt>
```

- Grammar: sonlu, boş olmayan bir kurallar kümesidir
- *Başlangıç sembolü (Start Symbol)*, gramerin nonterminal'lerinin özel bir öğesidir. (A *start symbol* is a special element of the nonterminals of a grammar)

Copyright © 2015 Pearson. All rights reserved.

3.3.1.2 BNF Kuralları

- Bir soyutlama (abstraction) (veya nonterminal sembol) birden fazla RHS'ye sahip olabilir.

```
<stmt> → <single_stmt>
       | begin <stmt_list> end
```

Copyright © 2015 Pearson. All rights reserved.

3.3.1.3 Lists Tanımlama

- Syntactic lists özyineleme kullanılarak tanımlanır.

```
<ident_list> → ident
             | ident, <ident_list>
```

- Türetme, start sembolüyle başlayan ve cümle (tüm terminal sembolleri) ile biten kuralların tekrarlı bir şekilde uygulanmasıdır.

Copyright © 2015 Pearson. All rights reserved.

3.3.1.4 Grammar Örneği

```
<program> → <stmts>
 <stmts> → <stmt> | <stmt> ; <stmts>
 <stmt> → <var> = <expr>
 <var> → a | b | c | d
 <expr> → <term> + <term> | <term> - <term>
 <term> → <var> | const
```

Copyright © 2015 Pearson. All rights reserved.

3.3.1.4 Bir Örnek Türetme (Derivation)

```
<program> => <stmts> => <stmt>
                     => <var> = <expr>
                     => a = <expr>
                     => a = <term> + <term>
                     => a = <var> + <term>
                     => a = b + <term>
                     => a = b + const
```

Copyright © 2015 Pearson. All rights reserved.

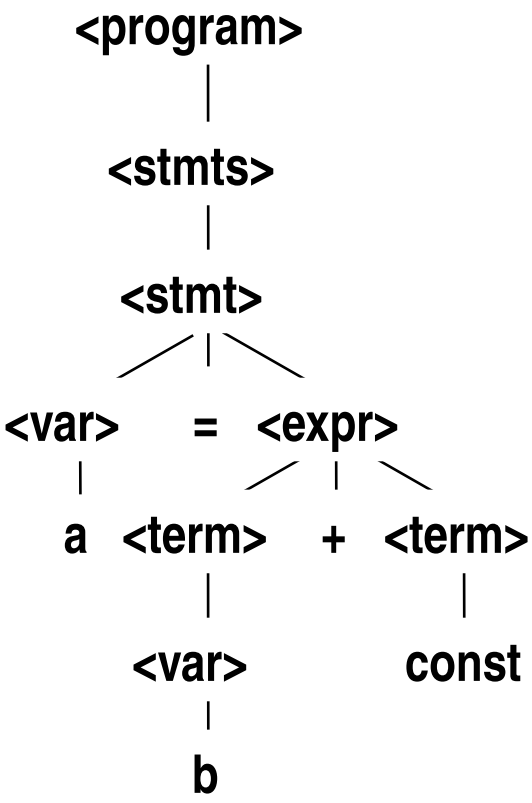
3.3.1.5 Türetmeler (Derivations)

- Türetmedeki her sembol dizisi bir cümle biçimidir (*sentential form*).
- Cümle (*sentence*), yalnızca terminal sembollerden oluşan cümle biçimidir(*sentential form*).
- En soldaki türetme, her cümle biçiminde en soldaki nonterminal terimin genişletilen terim olduğu türetmedir. (A *leftmost derivation* is one in which the leftmost nonterminal in each sentential form is the one that is expanded)
- Bir türetme ne en solda ne de en sağda olabilir. (A derivation may be neither leftmost nor rightmost.)

Copyright © 2015 Pearson. All rights reserved.

3.3.1.6 Parse Tree

- Bir türetmenin (derivation) hiyerarşik gösterimidir.



Copyright © 2015 Pearson. All rights reserved.

3.3.1.7 Ambiguity in Grammars (Gramerlerdeki Belirsizlikler)

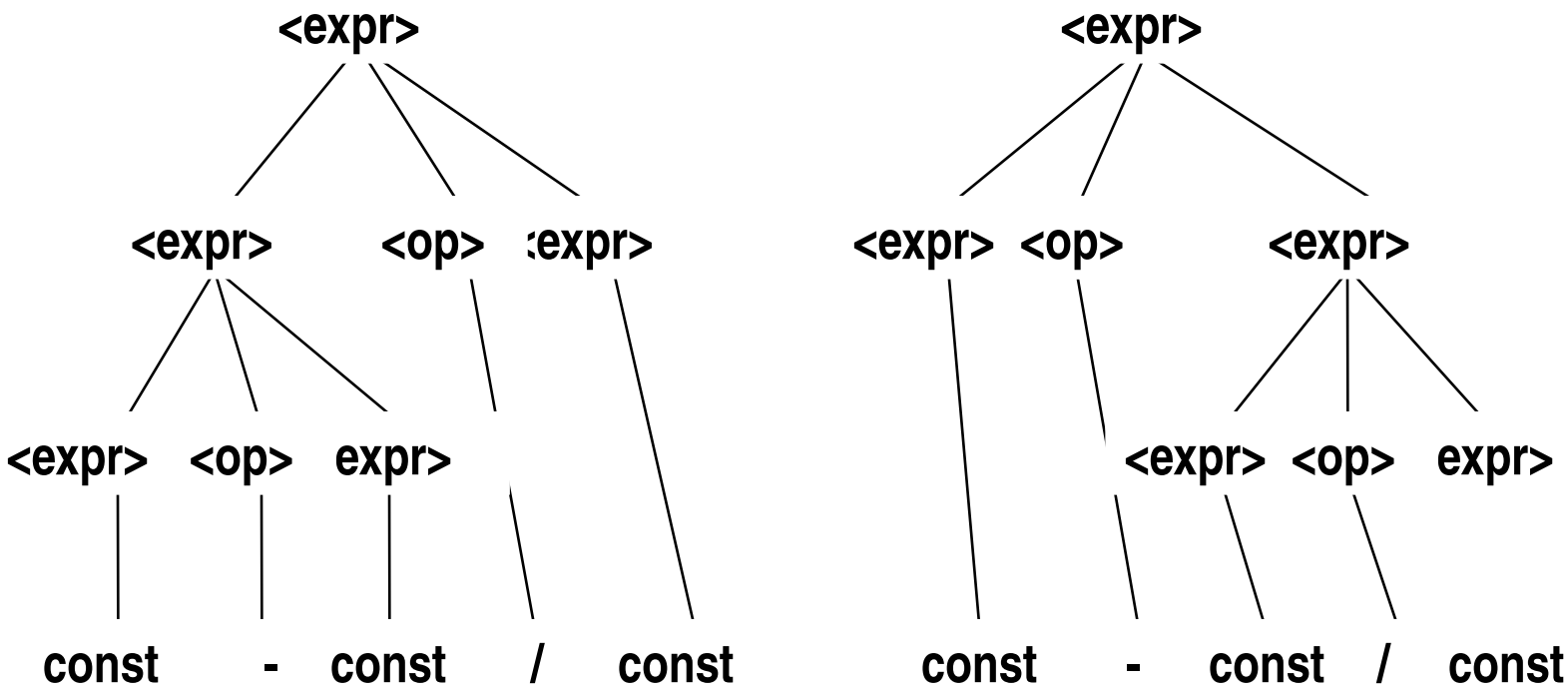
- Bir gramer, ancak ve ancak iki veya daha fazla farklı ayrıştırma ağacına sahip bir cümle biçimi oluşturursa belirsizdir.
  (A grammar is *ambiguous* if and only if it generates a sentential form that has two or more distinct parse trees)

Copyright © 2015 Pearson. All rights reserved.

3.3.1.7 Belirsiz Bir İfadenin Grameri (An Ambiguous Expression Grammar)

<expr> → <expr> <op> <expr> | const

<op> → / | -

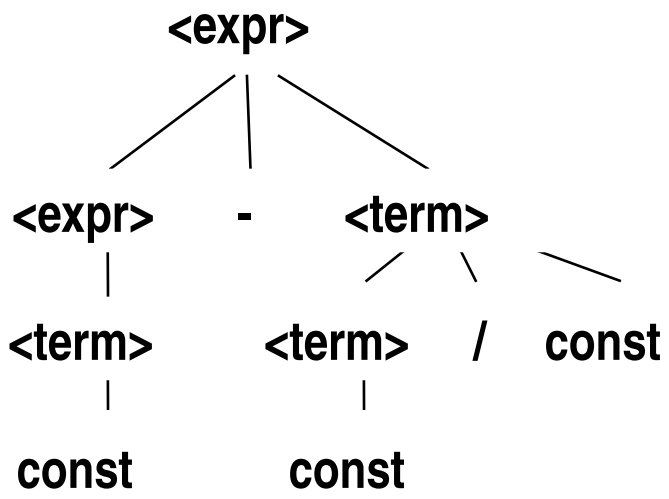


Copyright © 2015 Pearson. All rights reserved.

3.3.1.7 Belirsiz Olmayan Bir İfadenin Grameri (An Unambiguous Expression Grammar)

- Operatörlerin öncelik düzeylerini belirtmek için parse tree kullanırsak, belirsizlik oluşmaz.

```
<expr> → <expr> - <term>  |  <term>
<term> → <term> / const| const
```

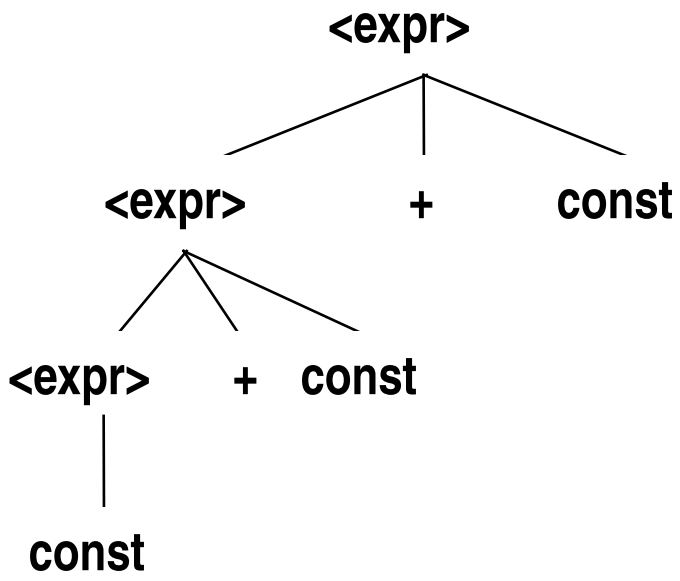


Copyright © 2015 Pearson. All rights reserved.

3.3.1.8 Operatörlerin Birleşebilirliği (Associativity of Operators)

- Operatör birleşebilirliği gramer ile de gösterilebilir.

```
<expr> -> <expr> + <expr> |  const  (ambiguous)
<expr> -> <expr> + const  |  const  (unambiguous)
```



Copyright © 2015 Pearson. All rights reserved.

3.3.1.9 Unambiguous Grammar for Selector

- Java `if-then-else` grammar

```
<if_stmt> -> if (<logic_expr>) <stmt>
           | if (<logic_expr>) <stmt> else <stmt>
```

Ambiguous!

- An unambiguous grammar for `if-then-else`

```
<stmt> -> <matched> | <unmatched>
<matched> -> if (<logic_expr>) <stmt>
             | a non-if statement
<unmatched> -> if (<logic_expr>) <stmt>
               | if (<logic_expr>) <matched> else
                    <unmatched>
```

Copyright © 2015 Pearson. All rights reserved.

3.3.2 Extended BNF

- İsteğe bağlı parçalar parantez içinde “[ ]” belirtilmiştir

  `<proc_call> -> ident [(<expr_list>)]`

- RHS'lerin alternatif parçaları parantez içine alınır ve dikey çubuklarla “|” ayrılır

  `<term> → <term> (+|-) const`

- Tekrarlar (0 veya daha fazla) “{ }” parantezleri içine yerleştirilir

  `<ident> → letter {letter|digit}`

Copyright © 2015 Pearson. All rights reserved.

3.3.2 BNF ve EBNF

- BNF

```
<expr> → <expr> + <term>
         | <expr> - <term>
         | <term>
 <term> → <term> * <factor>
         | <term> / <factor>
        | <factor>
```

- EBNF

```
<expr> → <term> {(+ | -) <term>}
 <term> → <factor> {(* | /) <factor>}
```

Copyright © 2015 Pearson. All rights reserved.

3.3.2 EBNF'deki Son Varyasyonlar

- Alternatif RHS'ler ayrı satırlara yazılır
- => yerine iki nokta üst üste kullanımı
- Opsiyonel parçalar için `opt` kullanımı
- Seçimler için `oneof` kullanımı

Copyright © 2015 Pearson. All rights reserved.

3.4 Statik Anlam Bilimi (Static Semantics)

- Anlamla ilgisi yok
- Context-free grammars (CFGs) programlama dillerinin tüm sözdizimini tanımlayamaz.
- Sorun teşkil eden yapı kategorileri:

  - Bağlamdan bağımsız (Context-free), ancak hantal (örneğin, ifadelerdeki operand türleri)

  - Bağlamdan bağımsız olmayan (Non-context-free) (örneğin, değişkenler kullanılmadan önce tanımlanmalıdır)

Copyright © 2015 Pearson. All rights reserved.

3.4.1 Attribute Grammars

- Attribute Grammars (AGs), CFG'lere eklemeler yaparak parse tree düğümleri hakkında bazı anlamsal bilgiler taşır.

- AG'lerin birincil değeri:
  - Statik semantik spesifikasyonu
  - Derleyici tasarımı (statik semantik kontrolü)

Copyright © 2015 Pearson. All rights reserved.

3.4.1 Attribute Grammars : Tanımlama

- Attribute grammar aşağıdaki eklemeleri içeren context-free grammar'dir G = (S, N, T, P):
  - Her gramer sembolü *x* için bir attribute değerleri kümesi *A(x)* vardır.
  - Her kural, kuraldaki nonterminallerin belirli attributes tanımlayan bir dizi işleve sahiptir.
  - Her kural, öznitelik (attribute) tutarlılığını kontrol etmek için (muhtemelen boş) bir dizi yüklem içerir.

Copyright © 2015 Pearson. All rights reserved.

3.4.1 Attribute Grammars: Tanımlama

- $X_0 \rightarrow X_1 \ldots X_n$ bir kural olsun
- $S(X_0) = f(A(X_1), \ldots, A(X_n))$ biçimindeki fonksiyonlar sentezlenmiş öznitelikleri (synthesized attributes) tanımlar
- *$I(X_j) = f(A(X_0), \ldots, A(X_n))$ formundaki fonksiyonlar, $i <= j <= n$ için*, miras alınan öznitelikleri (*inherited attributes) tanımlar*
- Başlangıçta, yapraklarda içsel öznitelikler (intrinsic attributes) vardır.

Copyright © 2015 Pearson. All rights reserved.

3.4.1.1 Attribute Grammars: Bir Örnek

- Syntax

  `<assign> -> <var> = <expr>`

  `<expr> -> <var> + <var> | <var>`

  `<var> A | B | C`

- `actual_type`: synthesized for `<var>` and `<expr>`
- `expected_type`: inherited for `<expr>`

Copyright © 2015 Pearson. All rights reserved.

3.4.1.1 Attribute Grammar (devam ediyor)

- Syntax kuralı: `<expr> → <var>[1] + <var>[2]`

  Semantic kuralları:

  `<expr>.actual_type ← <var>[1].actual_type`

  Predicate:

  `<var>[1].actual_type == <var>[2].actual_type`

  `<expr>.expected_type == <expr>.actual_type`

- Syntax kuralı: `<var> → id`

  Semantic kuralı:

  `<var>.actual_type ← lookup (<var>.string)`

Copyright © 2015 Pearson. All rights reserved.

3.4.1.1 Attribute Grammars (devam ediyor)

- Öznitelik değerleri (attribute values) nasıl hesaplanır?
  - Tüm öznitelikler miras alınmış olsaydı, ağaç yukarıdan aşağıya doğru sırayla (top-down order) süslenebilirdi.
  - Tüm öznitelikler sentezlenmiş olsaydı, ağaç aşağıdan yukarıya doğru sırayla (bottom-up order) süslenebilirdi.
  - Çoğu durumda, her iki tür öznitelik de kullanılır ve yukarıdan aşağıya (top-down) ve aşağıdan yukarıya (bottom-up) yaklaşımların bir kombinasyonu kullanılmalıdır.

Copyright © 2015 Pearson. All rights reserved.

3.4.1.1 Attribute Grammars (devam ediyor)

```
<expr>.expected_type ← inherited from parent

<var>[1].actual_type ← lookup (A)
<var>[2].actual_type ← lookup (B)
<var>[1].actual_type =? <var>[2].actual_type

<expr>.actual_type ← <var>[1].actual_type
<expr>.actual_type =? <expr>.expected_type
```

Copyright © 2015 Pearson. All rights reserved.

3.5 Anlam Bilimi (Semantics)

- Semantics'i tanımlamak için yaygın olarak kabul gören tek bir gösterim veya biçim yoktur.
- Semantics için metodoloji ve notasyon ihtiyacının çeşitli yönleri vardır:
  - Programcılar ifadelerin ne anlama geldiğini bilmelidir.
  - Derleyici yazarları, dil yapılarının ne yaptığını tam olarak bilmelidir.
  - Doğruluk kanıtları mümkün olacaktır.
  - Derleyici oluşturucular mümkün olabilir.
  - Tasarımcılar belirsizlikleri ve tutarsızlıkları tespit edebilirler.

Copyright © 2015 Pearson. All rights reserved.

3.5.1 Operasyonel Semantics

- Operasyonel Semantics
  - Bir programın anlamını, simüle edilmiş veya gerçek bir makinede ifadeleri yürütülerek açıklayabilirsiniz. Makinenin durumundaki değişiklik (“memory, registers”) ifadenin anlamını tanımlar.
- Yüksek seviyeli bir dilde operasyonel semantics kullanmak için sanal bir makine gereklidir.

Copyright © 2015 Pearson. All rights reserved.

3.5.1 Operasyonel Semantics

- Donanım tabanlı bir yorumlayıcı (pure interpreter) çok pahalı olurdu.
- Yazılım tabanlı bir yorumlayıcı (pure interpreter) da sorunlara sahiptir.
  - Belirli bir bilgisayarın ayrıntılı özellikleri, eylemlerin anlaşılmasını zorlaştırabilir.
  - Böyle bir semantic tanım, makineye bağlı olacaktır.

Copyright © 2015 Pearson. All rights reserved.

3.5.1 Operasyonel Semantics (Devam Ediyor)

- Daha iyi bir alternatif var: Tam bir bilgisayar simülasyonu
- Süreç:
  - Bir çevirici oluştur (kaynak kodunu idealize edilmiş bir bilgisayarın makine koduna çevirir)
  - İdealize edilmiş bilgisayar için bir simülatör oluştur
- Operasyonel semanticsin değerlendirilmesi:
  - İnformal kullanımda iyidir (dil kılavuzları vb.)
  - Formel olarak kullanıldığında (örneğin, VDL) son derece karmaşık olan bu dil, PL/I'nin semantiğini tanımlamak için kullanıldı.

Copyright © 2015 Pearson. All rights reserved.

3.5.1 Operasyonel Semantics (Devam Ediyor)

- Operasyonel semantics kullanım alanları:
  - Dil kılavuzları ve ders kitapları
  - Programlama dilleri öğretimi

- Operasyonel Semantics'in iki farklı kullanım düzeyi vardır:
  - Doğal operasyonel semantics
  - Yapısal operasyonel semantics

- Değerlendirme
  - İnformel kullanımda iyidir (dil kılavuzları vb.)
  - Formel olarak kullanıldığında son derece karmaşık (örneğin, VDL)

Copyright © 2015 Pearson. All rights reserved.

3.5.2 Denotasyonel Semantics

- Özyinelemeli fonksiyon teorisine dayalıdır
- En soyut semantics tanımlama yöntemi
- İlk olarak Scott ve Strachey (1970) tarafından geliştirilmiştir

Copyright © 2015 Pearson. All rights reserved.

3.5.2 Denotasyonel Semantics (Devam Ediyor)

- Bir dil için denotasyonel bir spesifikasyon oluşturma süreci:

  - Her dil öğesi için bir matematiksel nesne tanımla
    - Dil öğelerinin örneklerini karşılık gelen matematiksel nesnelerin örneklerine eşleyen bir fonksiyon tanımla
- Dil yapılarının anlamı, yalnızca programın değişkenlerinin değerleri tarafından tanımlanır

Copyright © 2015 Pearson. All rights reserved.

3.5.2 Denotasyonel Semantics: program durumu

- Bir programın durumu, tüm geçerli değişkenlerinin değerleridir.

$$s = \{\langle i_1, v_1\rangle, \langle i_2, v_2\rangle, \ldots, \langle i_n, v_n\rangle\}$$

- VARMAP, bir değişken adı ve durumu verildiğinde, değişkenin geçerli değerini döndüren bir fonksiyon olsun.

$$\text{VARMAP}(i_j, s) = v_j$$

Copyright © 2015 Pearson. All rights reserved.

3.5.2.1 Decimal Numbers

```
<dec_num> →  '0' | '1' | '2' | '3' | '4' | '5' |
             '6' | '7' | '8' | '9' |
             <dec_num> ('0' | '1' | '2' | '3' |
                        '4' | '5' | '6' | '7' |
                        '8' | '9')
```

M_{dec}('0') = 0, M_{dec} ('1') = 1, …, M_{dec} ('9') = 9

M_{dec} (<dec_num> '0') = 10 * M_{dec} (<dec_num>)

M_{dec} (<dec_num> '1’) = 10 * M_{dec} (<dec_num>) + 1

…

M_{dec} (<dec_num> '9') = 10 * M_{dec} (<dec_num>) + 9

Copyright © 2015 Pearson. All rights reserved.

3.5.2.1 İfadeler (Expressions)

- Z üzerine ifadelerin eşlenmesini yap (Z ∪ {error})
- İfadelerin ondalık sayılar, değişkenler veya bir aritmetik operatör ve iki operand içeren binary ifadeler olduğunu varsayıyoruz; her bir operand bir ifade olabilir.

Copyright © 2015 Pearson. All rights reserved.

3.5.2.1 İfadeler (Expressions)

```
Me(<expr>, s) Δ=
    case <expr> of
      <dec_num> => Mdec(<dec_num>, s)
      <var> =>
           if VARMAP(<var>, s) == undef
                then error
                else VARMAP(<var>, s)
     <binary_expr> =>
          if (Me(<binary_expr>.<left_expr>, s) == undef
                OR Me(<binary_expr>.<right_expr>, s) =
                                 undef)
              then error
           else
           if (<binary_expr>.<operator> == '+' then
              Me(<binary_expr>.<left_expr>, s) +
                     Me(<binary_expr>.<right_expr>, s)
           else Me(<binary_expr>.<left_expr>, s) *
               Me(<binary_expr>.<right_expr>, s)
        ...
```

Copyright © 2015 Pearson. All rights reserved.

3.5.2.2 Atama İfadeleri (Assignment Statements)

- Durum kümelerini U {error} durum kümelerine haritalar

M_a(x := E, s) Δ=
    if M_e(E, s) == error
        then error
        else s′ =
  {<i_1,v_1′>,<i_2,v_2′>,...,<i_n,v_n′>},
                where for j = 1, 2, ..., n,
                    if i_j == x
                      then v_j′ = M_e(E, s)
                      else v_j′ = VARMAP(i_j, s)

Copyright © 2015 Pearson. All rights reserved.

3.5.2.3 Mantıksal Ön Test Döngüleri

- Durum kümelerini U {error} durum kümelerine haritalar

```
Ml(while B do L, s) Δ=
    if Mb(B, s) == undef
        then error
        else if Mb(B, s) == false
            then s
            else if Msl(L, s) == error
                  then error
                  else Ml(while B do L, Msl(L, s))
```

Copyright © 2015 Pearson. All rights reserved.

3.5.2.3.1 Döngünün Anlamı

- Döngünün anlamı, döngüdeki ifadelerin öngörülen sayıda çalıştırılmasından sonra, herhangi bir hata olmadığı varsayılarak program değişkenlerinin değeridir.
- Özünde, döngü yinelemeden (iteration) tekrarlama (recursion) dönüştürülmüştür; burada tekrarlamalı kontrol, diğer tekrarlamalı durum eşleme işlevleri tarafından matematiksel olarak tanımlanmıştır.

- Tekrarlama, yinelemeyle karşılaştırıldığında matematiksel titizlikle tanımlanması daha kolaydır.

Copyright © 2015 Pearson. All rights reserved.

3.5.2.4 Denotasyonel Semantics'in Değerlendirilmesi

- Programların doğruluğunu kanıtlamak için kullanılabilir
- Programlar hakkında düşünmek için titiz bir yol sağlar
- Dil tasarımına yardımcı olabilir
- Derleyici oluşturma sistemlerinde kullanılmıştır
- Karmaşıklığı nedeniyle dil kullanıcıları için pek kullanışlı değildir

Copyright © 2015 Pearson. All rights reserved.

3.5.3 Aksiyomatik Semantics

- Formel mantığa (yüklem hesabı) dayalı
- Orijinal amaç: formel program doğrulaması
- Dil içindeki her ifade türü için aksiyomlar veya çıkarım kuralları tanımlanır (mantık ifadelerinin daha resmi mantık ifadelerine dönüştürülmesine izin vermek için)
- Mantıksal ifadelere *iddialar (assertions)* denir

Copyright © 2015 Pearson. All rights reserved.

3.5.3 Aksiyomatik Semantics (Devam Ediyor)

- Bir ifadeden önceki bir iddia *(ön koşul)*, yürütme sırasında o noktada doğru olan değişkenler arasındaki ilişkileri ve kısıtlamaları belirtir
- Bir ifadeyi takip eden bir iddia bir son koşuldur (*postcondition*)
- *En zayıf ön koşul (A weakest precondition)*, son koşulu garanti edecek en az kısıtlayıcı ön koşuldur

Copyright © 2015 Pearson. All rights reserved.

3.5.3.1 Aksiyomatik Semantics Form

- Pre-, post form: `{P} statement {Q}`

- Örneğin
  - `a = b + 1  {a > 1}`
  - Olası bir ön koşul (precondition): `{b > 10}`
  - En zayıf ön koşul (precondition): `{b > 0}`

Copyright © 2015 Pearson. All rights reserved.

3.5.3.2 Program Kanıt Süreci

- Tüm programın son koşulu istenen sonuçtur
  - Programı ilk ifadeye kadar geriye doğru çalışın. İlk ifadedeki ön koşul, program spesifikasyonuyla aynıysa, program doğrudur.

Copyright © 2015 Pearson. All rights reserved.

3.5.3.3 Aksiyomatik Semantics: Atama

- Atama ifadeleri için bir aksiyom:
  (x = E): $\{Q_{x->E}\}$ x = E {Q}

- Sonuç Kuralı:

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q \Rightarrow Q'}{\{P'\}\ S\ \{Q'\}}$$

Copyright © 2015 Pearson. All rights reserved.

3.5.3.4 Aksiyomatik Semantics: Diziler

- S1; S2 biçimindeki diziler için bir çıkarım kuralı

  {P1} S1 {P2}

  {P2} S2 {P3}

$$\frac{\{P1\}\ S1\ \{P2\},\ \{P2\}\ S2\ \{P3\}}{\{P1\}\ S1;\ S2\ \{P3\}}$$

Copyright © 2015 Pearson. All rights reserved.

3.5.3.5 Aksiyomatik Semantics: Seçim

- Seçim için bir çıkarım kuralı
  - **`if`** B **`then`** S1 **`else`** S2

$$\frac{\{B \text{ and } P\}\ S1\ \{Q\},\ \{(\text{not } B) \text{ and } P\}\ S2\ \{Q\}}{\{P\}\ \textbf{if}\ B\ \textbf{then}\ S1\ \textbf{else}\ S2\ \{Q\}}$$

Copyright © 2015 Pearson. All rights reserved.

3.5.3.6 Aksiyomatik Semantics: Döngüler

- Mantıksal ön test döngüleri için bir çıkarım kuralı

  {P} **`while`** B **`do`** S **`end`** {Q}

$$\frac{(\text{I and B})\ \text{S}\ \{I\}}{\{I\}\ \text{while B do S}\ \{\text{I and (not B)}\}}$$

burada I döngü değişmezidir (tümevarım hipotezi)

Copyright © 2015 Pearson. All rights reserved.

3.5.3.7 Aksiyomatik Semantics: Aksiyomlar

- Döngü değişmezinin özellikleri: Aşağıdaki koşulları sağlamam gerekir:
  - P => I -- döngü değişmezi başlangıçta doğru olmalıdır
  - {I} B {I} -- Boolean'ın değerlendirilmesi I'nin geçerliliğini değiştirmemelidir
  - {I and B} S {I} -- Döngünün gövdesini çalıştırarak I değişmez
  - (I and (not B)) => Q -- eğer I doğruysa ve B yanlışsa, Q ima edilir
  - The loop terminates -- kanıtlanması zor olabilir

Copyright © 2015 Pearson. All rights reserved.

3.5.3.8 Döngü Değişmezi

- Döngü değişmezi I, döngü son koşulunun zayıflatılmış halidir ve aynı zamanda bir ön koşuldur.
- Döngü başlamadan önce tatmin olacak kadar zayıf olmalıyım, ancak döngü çıkış koşulu ile birleştirildiğinde, son koşulun doğruluğunu zorlayacak kadar güçlü olmalıdır.

Copyright © 2015 Pearson. All rights reserved.

3.5.3.9 Aksiyomatik Semantics'in Değerlendirilmesi

- Bir dildeki tüm ifadeler için aksiyomlar veya çıkarım kuralları geliştirmek zordur
- Doğruluk kanıtları için iyi bir araçtır ve programlar hakkında akıl yürütmek için mükemmel bir çerçevedir, ancak dil kullanıcıları ve derleyici yazarları için o kadar yararlı değildir
- Bir programlama dilinin anlamını tanımlamadaki faydası dil kullanıcıları veya derleyici yazarları için sınırlıdır

Copyright © 2015 Pearson. All rights reserved.

3.5.4 Denotasyonel Semantics vs Operasyonel Semantics

- Operasyonel Semantics'te, durum değişiklikleri kodlanmış algoritmalar tarafından tanımlanır
- Operasyonel Semantics'te, durum değişiklikleri titiz matematiksel fonksiyonlarla tanımlanır

Copyright © 2015 Pearson. All rights reserved.

3.5.5 Özet

- BNF ve bağlamdan bağımsız gramer (context-free grammars) eşdeğer meta dillerdir.
  - Programlama dillerinin söz dizimini (syntax) tanımlamak için oldukça uygundur
- Attribute Grammar, bir dilin hem söz dizimini (syntax) hem de anlam bilimini (semantics) tanımlayabilen tanımlayıcı bir biçimciliktir
- Semantics tanımlamanın üç temel yöntemi:
  - Operasyonel, Aksiyomatik, Denotasyonel

Copyright © 2015 Pearson. All rights reserved.